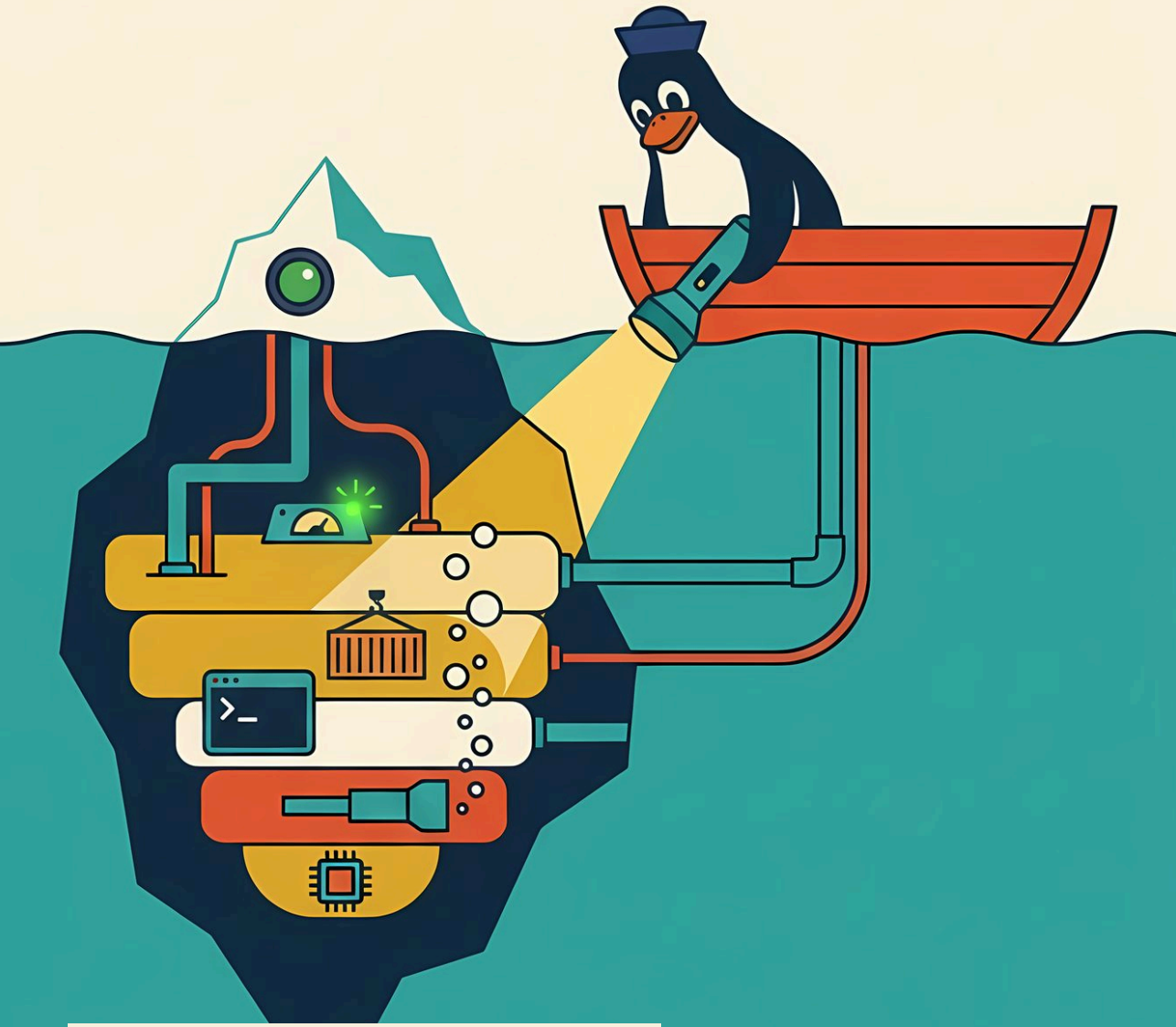
CAPÍTULO DE MUESTRA

NO PODÉS DEFENDER LO QUE NO ENTENDÉS

Linux, las abstracciones que fallan y el camino a la ciberseguridad



VALENTÍN TORASSA COLOMBERO



1

TODA ABSTRACCIÓN FILTRA

DÓNDE ESTAMOS

- Tablero y nube
- Orquestador
- Contenedor
- Espacio de usuario
- Kernel
- Hardware

Este capítulo arranca con un incidente que reconoce cualquiera que haya trabajado con sistemas en producción. Termina con la idea que vuelve en todo el libro, que es tener un modelo mental de lo que cada abstracción esconde, sin necesidad de memorizar el kernel.

En el medio vas a ver de dónde sale esa idea, por qué tiene más de veinte años y cómo se ve una fuga en tu propia terminal, con un script de seis líneas que corrés como usuario común. Los ejemplos se probaron en Debian 13, y `fuga.sh`, también en Ubuntu 24.04 y en Fedora 43, con el mismo resultado.

El tablero en verde

Hay un tipo de incidente que me tocó vivir muchas veces, y que a esta altura reconozco apenas empieza. La versión más chica me pasó con mi propia cuenta de LinkedIn. Le pedí a su API las métricas de trece posts míos y me contestó con un 200, que quiere decir que todo salió bien, y cero likes en cada uno. No era que nadie los hubiera visto. El token no tenía permiso para leer esas métricas, y en vez de un 403, el error de acceso denegado, la API me devolvía ceros.

En producción, la escena es siempre parecida. Alguien avisa que el servicio no responde, o que responde tan lento que da lo mismo. Abrís el tablero de monitoreo y está todo en verde. El contenedor figura como `healthy`, el uso de CPU es normal y la consola de la nube dice que la instancia está sana. Y aun así, del otro lado, el sistema no funciona.

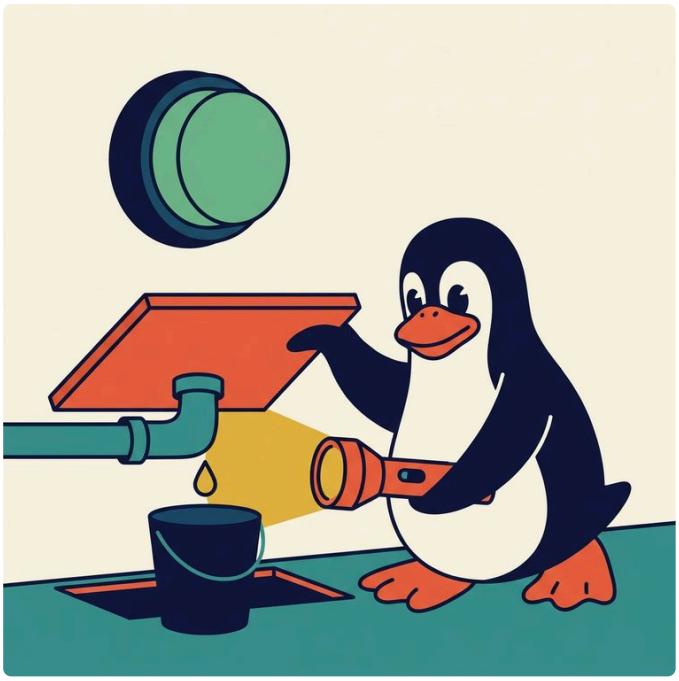
Lo primero que hacemos casi todos es desconfiar del aviso. Lo segundo, refrescar el tablero. Lo tercero, reiniciar algo y esperar que alcance. En ese momento casi nadie mira lo que está abajo de todo eso, que es el host, el kernel y los procesos en ejecución.

Y es razonable que no lo hagamos, porque para eso existe el tablero. Te promete que no tenés que mirar abajo. El orquestador mantiene vivo el contenedor y la nube se ocupa de la máquina. El tablero te resume todo en un color, y si está en verde, está bien.

El problema es que esa promesa se cumple casi siempre, y con un «casi siempre» te podés quedar solo frente al incidente a las tres de la mañana.

Cuando el tablero está en verde y el sistema no anda, el tablero cuenta una verdad a medias, porque muestra lo que sabe la capa de arriba. El problema está en una capa que el tablero no mira. Puede ser un proceso que el kernel mató por pasarse de memoria. Puede ser una regla de red que deja pasar el tráfico entre dos contenedores pero no desde el host. Puede ser un disco que se quedó sin inodos aunque todavía tenga espacio libre. Nada de eso aparece en el tablero, ni en verde ni en rojo.

La abstracción no mintió. Filtró.



El motor y la luz del tablero

Para mí, la mejor forma de entenderlo es pensar en un auto. Para manejar no necesitás saber cómo funciona el motor. La gracia del auto es que te da un volante, dos pedales y una palanca, y te esconde todo lo demás.

Pero el día que se prende una luz en el tablero, tenés que saber, al menos, de qué parte del auto te está hablando, aunque no sepas arreglar el motor. Con esa sola distinción ya decidís si podés seguir manejando, si tenés que parar o a quién tenés que llamar.

Con los sistemas pasa lo mismo. Cuando algo falla de una forma que la abstracción no sabe explicar, la diferencia entre resolverlo en diez minutos y pasar la noche en vela es saber de qué capa te está hablando el síntoma.

La ley de las abstracciones con fugas

Esto no lo descubrí yo. En noviembre de 2002, Joel Spolsky publicó en su blog un artículo que se llama «The Law of Leaky Abstractions», la ley de las

abstracciones con fugas. Lo escribió como un texto de opinión, y aun así, para mí, envejeció mejor que muchos papers.

La ley entra en una línea: «toda abstracción no trivial, en alguna medida, tiene fugas».[1]

Uno de los ejemplos de Spolsky sigue vigente. TCP te promete que los datos llegan completos y en orden sobre una red, IP, que no garantiza ninguna de las dos cosas. Y lo cumple, hasta que alguien corta un cable o un equipo del medio se satura. Ahí la red de abajo se ve a través de TCP, en forma de conexiones que se cuelgan o que tardan una eternidad.

TCP está entre las mejores ideas que tuvo la informática, y para mí filtra por eso mismo. Una abstracción trivial esconde tan poco que casi no tiene nada para dejar escapar. Una que vale la pena, como TCP, esconde mucho, y cualquier parte de eso puede asomarse algún día.

Trabajo, no aprendizaje

Hay otra frase del mismo artículo que me parece todavía más importante que la ley. Spolsky dice que las abstracciones nos ahorran tiempo de trabajo, pero no tiempo de aprendizaje.[2]

Docker te ahorra horas de instalar dependencias y de armar el mismo entorno en cada máquina, y por eso lo usamos todos. Pero no te ahorra entender qué es un contenedor, y mientras todo funciona eso no se nota. El día que filtra, es lo primero que vas a necesitar.

Las capas de un sistema Linux

Para hablar de capas hace falta un mapa. La Figura 1-1 es el que vamos a usar en todo el libro. Se lee de abajo hacia arriba, porque así está construido. Cada capa usa la de abajo y le esconde detalles a la de arriba.

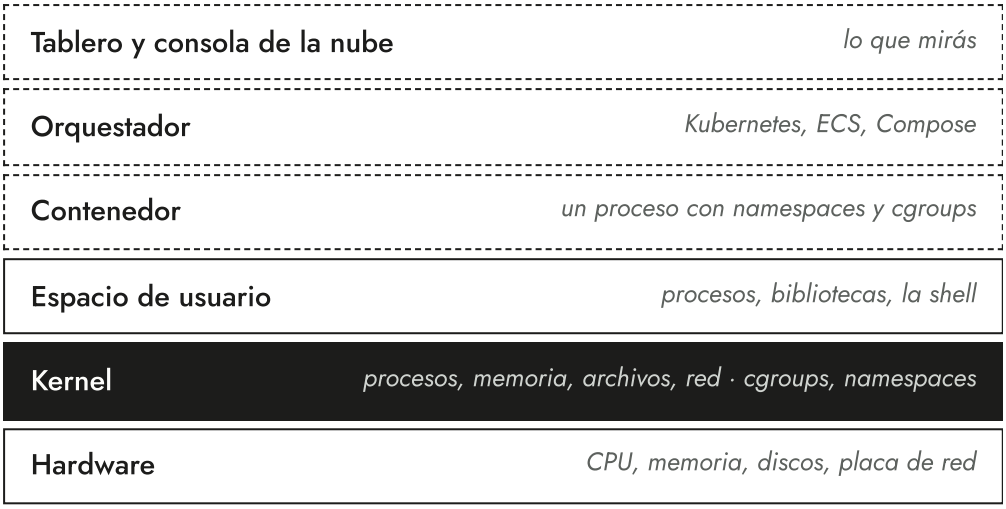


Figura 1-1: *Las capas de un sistema Linux, de abajo hacia arriba. El tablero mira las de arriba; la fuga de este capítulo vive en el kernel.*

Abajo de todo está el **hardware**. Arriba está el **kernel**, el núcleo del sistema operativo, que es el único programa que habla con el hardware de forma directa. Todo lo demás, los procesos que lanzás, sus bibliotecas y la shell, vive en el **espacio de usuario**. Un proceso del espacio de usuario no puede tocar el hardware, y cuando necesita memoria, un archivo o la red, se lo pide al kernel.

Las tres capas de arriba son las que usamos todos los días, y quizás te sorprenda que tu programa, adentro de ellas, siga hablando con el mismo kernel, sin nada en el medio. Un contenedor es un proceso común del espacio de usuario. El kernel le muestra una vista recortada del sistema (eso son los *namespaces*, los espacios de nombres) y le pone límites de recursos (eso son los *cgroups*, los grupos de control). El orquestador decide qué contenedores corren y dónde. El tablero te resume lo que el orquestador sabe.

Fijate qué pasa con el primero de los casos del principio, el proceso que el kernel mató por pasarse de memoria. El tablero mira las capas de arriba, y el límite que lo mató lo puso un cgroup, que es del kernel. El síntoma y la causa están en capas distintas, y por eso el tablero sigue en verde.

Una fuga, de cerca

Hasta acá fue todo teoría, así que vamos a provocar una fuga a propósito, en tu máquina, como usuario común y sin Docker. El Listado 1-1 es un script de seis líneas.

```
   #!/bin/sh
❶  python3 -c "b = bytearray(200 * 1024 * 1024)"
❷  echo "python terminó con: $?"
❸  cg=/sys/fs/cgroup$(cut -d: -f3 /proc/self/cgroup)
❹  cat "$cg/memory.max"
❺  grep oom "$cg/memory.events"
```

Listado 1-1: `fuga.sh`*, un proceso que pide más memoria de la que le van a dar*

En ❶, Python reserva 200 MiB de una sola vez. En ❷, la shell imprime cómo terminó Python con la variable `$?`, que guarda el código de salida del último comando. En ❸ arma la ruta del cgroup en el que corre el script, que el kernel publica en `/proc/self/cgroup`. Con esa ruta, en ❹ lee el límite de memoria que el kernel le aplica (`memory.max`) y en ❺ lee el contador de eventos de memoria de ese mismo cgroup. Si `python3` no está, en Debian se instala con `sudo apt install python3`.

Para correrlo con un límite usamos `systemd-run`, que crea un cgroup nuevo, le pone las propiedades que le pidas y lanza el comando adentro, y el Listado 1-2 muestra la corrida.

```
   $ systemd-run --user --scope -q -p MemoryMax=50M \
       -p MemorySwapMax=0 sh fuga.sh
❶  Killed
❷  python terminó con: 137
❸  52428800
❹  oom 1
   oom_kill 1
   oom_group_kill 0
```

Listado 1-2: *El kernel mata a Python y el script lee por qué*

Como el comando no entra en un renglón, la barra invertida lo corta y sigue en el de abajo. `--user` lo corre como tu usuario, sin root, y `MemoryMax=50M` es el límite. `MemorySwapMax=0` le prohíbe usar swap, la parte del disco que el kernel usa como memoria de reserva. Python pide 200 MiB y le dieron 50.

La salida se lee igual que el script. En ❶, la shell avisa que Python murió de golpe, y en ❷ da el código de salida, 137. En ❸ está el límite que aplicó el kernel, 52.428.800 bytes, que son 50 MiB. En ❹ está su contador, con un evento de falta de memoria (`oom`) y un proceso muerto por el *OOM killer* (`oom_kill`), la parte del kernel que elige a quién matar cuando la memoria no alcanza. La Figura 1-2 ordena esa secuencia por capa.

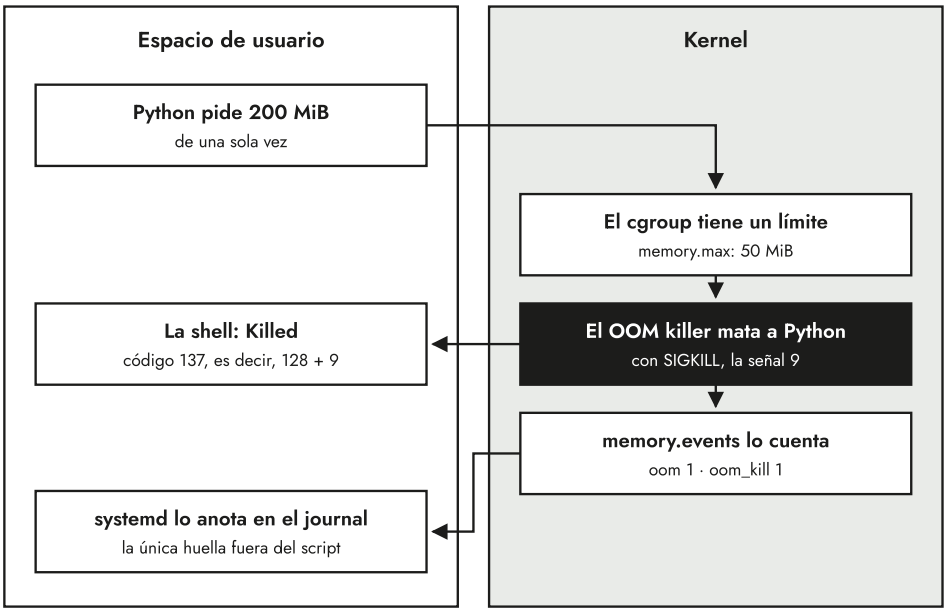


Figura 1-2: *La fuga, paso a paso. El límite y la muerte pasan en el kernel; arriba llegan un 137 y el journal.*

NOTA

El 137 sale de una cuenta. Cuando una señal termina un proceso, la shell informa 128 más el número de la señal, y la 9 es `SIGKILL`*, la única que un proceso no puede atrapar ni ignorar. Por eso Python no alcanzó a decir nada, ni un error, ni una traza, ni un mensaje.*

▲ **ADVERTENCIA**

Si corrés el Listado 1-2 varias veces, muchas pueden cortarse antes, con un `Terminated` *y sin los contadores; en algunas máquinas, la mayoría. Si te pasa, tu máquina anda bien. Volvé a correrlo hasta que salga completo. Interviene otra capa, systemd, que encontramos en el capítulo 12, cuando convirtamos esta corrida en evidencia.*

Ahora mirá la máquina desde arriba, como la miraría un tablero. El Listado 1-3 es `free`, corrido en la misma máquina segundos después.

```
$ free -h
               total        used        free  [...]   available
Mem:            60Gi        21Gi       2.6Gi  [...]        38Gi
Swap:           49Gi       6.6Gi        42Gi
```

Listado 1-3: *Desde arriba, sobran 38 GiB de memoria*

Desde arriba, sobran 38 GiB. Desde adentro del cgroup, no había 200 MiB. Las dos cosas son ciertas a la vez, y la Figura 1-3 las pone una al lado de la otra. Yo lo resumo diciendo que `free` confunde y `memory.max` manda.

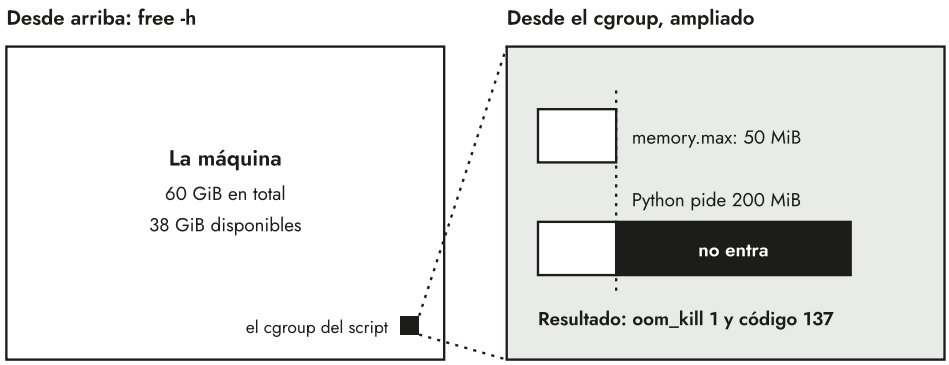


Figura 1-3: *La misma memoria, desde arriba y desde el cgroup. A la máquina le sobran 38 GiB; a Python le dieron 50 MiB.*

La única huella que queda fuera del script está en el *journal* (el registro de systemd), como muestra el Listado 1-4.

```
$ journalctl --user --since "-2min" | grep -i oom
[...] systemd[1932]: [...] has been killed by the OOM killer.
[...] systemd[1932]: [...] Failed with result 'oom-kill'.
```

Listado 1-4: *La única huella de la muerte de Python, en el journal*

Un contenedor con `docker run --memory 50m` usa el mismo mecanismo, porque Docker le pide al kernel un límite en el mismo archivo, `memory.max`. En el capítulo 8, un contenedor sin swap que pide más que su límite muere con 137 y Docker anota `OOMKilled: true`. La diferencia es cuántas capas hay entre vos y el kernel cuando pasa.

▲ **ADVERTENCIA**

`fuga.sh` *es inofensivo con el límite puesto. Si cambiás 200 por un número cercano a la memoria total de tu máquina y lo corrés sin* `systemd-run`*, el que actúa es el OOM killer del sistema entero, y no siempre elige al proceso que esperás.*

MI PIEZA DE SOFTWARE FAVORITA

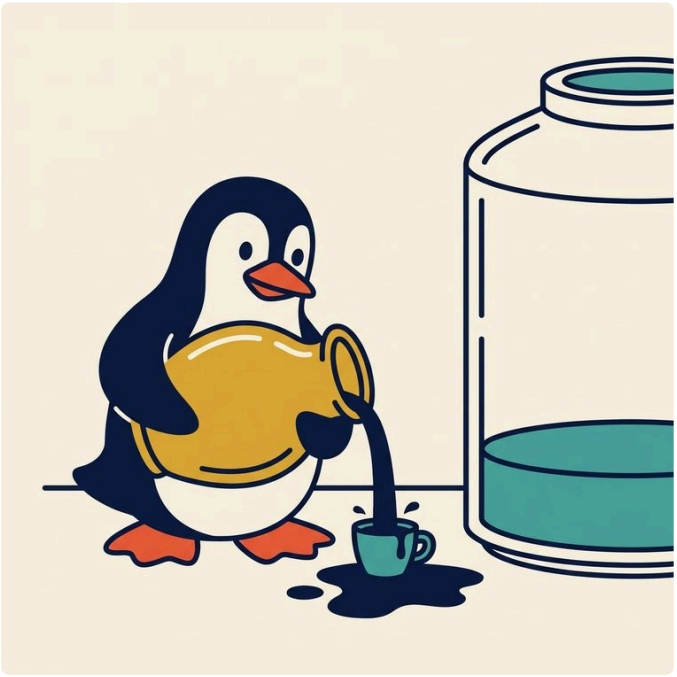
Unix tiene un videojuego caro en su origen. En 1968 y 1969, los Laboratorios Bell se estaban yendo de Multics, un proyecto de sistema operativo enorme que compartían con otras instituciones. Ken Thompson había escrito un juego, Space Travel, que simulaba el sistema solar. En la computadora grande del laboratorio, una GE 635, la imagen se movía a los saltos y cada partida costaba unos 75 dólares de tiempo de CPU.

Thompson encontró una PDP-7 que casi nadie usaba y pasó el juego ahí con Dennis Ritchie. En esa máquina, sobre un sistema de archivos que había diseñado con Ritchie y R. H. Canaday, Thompson armó lo que después se llamó Unix.

> Para mí, Unix es especial, y es mi pieza de software favorita porque las abstracciones que diseñaron entonces siguen siendo universales. Linux las replicó y las fue mejorando, y más de cincuenta años después seguimos usándolas como la madre de muchas de las cosas que construimos hoy. Los procesos, los archivos, los usuarios y los permisos que diseñaron en esa PDP-7 siguen abajo de tu contenedor.

No hace falta memorizar el kernel

Puede que este capítulo te haya dejado la sensación de que tenés que saberte el kernel de memoria para trabajar en sistemas. Alcanza con un modelo mental de lo que cada abstracción esconde, y en el ejemplo de este capítulo ese modelo es corto. Un contenedor es un proceso con un límite que pone el kernel.



De ese modelo salen tres hábitos, que vuelven al final del libro:

1. **Nombrá la capa del problema.** Antes de buscar la solución, decí en voz alta en qué capa de la Figura 1-1 creés que está.
2. **Usá una herramienta de abajo antes de abrir un ticket.** Un `cat` de `memory.events` te dice en un segundo lo que el tablero no te va a decir nunca.
3. **Diseñá para la fuga.** Poné límites y timeouts como si la fuga fuera inevitable, porque lo es.

Esos hábitos te dejan seguir usando las abstracciones sin confiar en ellas a ciegas.

En la era de la IA, para mí importa más que nunca no subestimar lo que corre abajo de nuestros sistemas. Cuando lo de abajo falla, el problema no se resuelve con la abstracción en la que estamos trabajando. Y creo que el discurso de que todos tenemos que estar en la capa de arriba le trae a nuestra disciplina problemas éticos y morales, que tenemos que resolver entendiendo que todas las capas importan.

Cómo lo mostrás

Si estás buscando tu primer trabajo en sistemas o en seguridad, este capítulo ya te da algo para mostrar. Corré `fuga.sh`, guardá la salida y escribí una página que cuente qué provocaste, cómo lo explicaste con el límite y el contador del kernel, y qué cambiarías en un servicio real. Una página así dice más que «conocimientos de Linux» en un CV, porque se puede comprobar.

LO QUE VISTE

- **Toda abstracción no trivial filtra.** Cuanto más esconde, más tiene para dejar escapar.
- **Síntoma y causa, en capas distintas.** El tablero mira arriba; el límite que mató a Python lo puso un cgroup del kernel.
- `free` **confunde,** `memory.max` **manda.** A la máquina le sobraban 38 GiB y Python murió igual, con un 137 y sin mensaje. `grep oom memory.events`

Probalo

1. Corré `fuga.sh` con `MemoryMax=300M`. ¿Qué cambia en la salida y por qué?
2. Sacá `-p MemorySwapMax=0` y volvé a correrlo con 50M. En la máquina de las pruebas, Python sobrevivió. Terminó con 0 y `oom_kill` quedó en 0. ¿Adónde fue la memoria que faltaba? Agregá al final de `fuga.sh` la línea `cat "$cg/memory.swap.peak"`, que guarda lo máximo que el cgroup usó de swap, y pensá qué vería un usuario de ese servicio. Probablemente vería lentitud, y ningún error. Si tu máquina no tiene swap (la fila `Swap` de `free -h` en cero), Python va a morir igual que en el Listado 1-2.
3. Elegí el último problema raro que viste en una computadora, propia o del trabajo, y ubicalo en la Figura 1-1. ¿El síntoma y la causa estaban en la misma capa?

Si nunca abriste una terminal, empezá por el lab «Moverte en la terminal: cd sin perderte» de Open Security Labs (https://securitylabs.valentorassa.com). Son quince minutos, y el resto del libro da por hecho que sabés dónde estás y cómo llegar a un archivo.

Lo que viene

En este capítulo cruzamos de golpe desde el tablero hasta el kernel. En el próximo vamos a hacer el mismo viaje despacio, capa por capa, desde que escribís `ls` hasta que el kernel lee el disco. Y `strace` te va a dejar ver cada pedido en el camino.

NOTAS

CAPÍTULO 1. TODA ABSTRACCIÓN FILTRA

1. «All non-trivial abstractions, to some degree, are leaky.» Joel Spolsky, «The Law of Leaky Abstractions», *Joel on Software*, 11 de noviembre de 2002.
2. «So the abstractions save us time working, but they don't save us time learning.» Mismo artículo.

SEGUÍ LEYENDO

Esto fue el capítulo 1 de *No podés defender lo que no entendés*, tal como sale en el libro impreso. El libro tiene 12 capítulos en tres partes, más un prefacio y un epílogo.

EL LIBRO

El libro está en preparación.

Anotate para recibir el aviso cuando estén confirmados fecha, precio y dónde conseguirlo: https://valentorassa.com/libro?ref=muestra-pdf.

CONTENIDO

Prefacio. Borré mi compu

PARTE I. EL MAPA

1. Toda abstracción filtra (esta muestra)
2. Del comando al kernel
3. Programa, proceso, thread
4. Quién actúa, y con qué permiso
5. Todo es un archivo
6. Arranque, servicios y logs

PARTE II. LAS FUGAS

7. Una request no es magia
8. Un contenedor no es una VM
9. La nube es la computadora de otro
10. Agentes: el mismo problema con otra cara

PARTE III. LA EVIDENCIA

11. Debian, o el permiso para mirar
12. La experiencia se demuestra

Epílogo. Diseñá para la fuga

LOS EJEMPLOS Y LOS LABS

Los scripts de esta muestra están en `ejemplos/cap01/` de https://github.com/ValentinTorassa/VT-Book-Examples, con los del resto del libro, tal como salen en el papel. Los labs de cada capítulo son gratis y están en Open Security Labs, https://securitylabs.valentorassa.com.

Valentín Torassa Colombero trabaja en ciberseguridad en Rosario y hace VT Security, un canal sobre Linux y seguridad. Este libro nació de su charla «Abstraction Leaks», en DebConf26.

© 2026, Valentín Torassa Colombero. Podés compartir este PDF completo y sin cambios.